

Schema Design Document

PostgreSQL schema boundaries, table designs, enumerations, and data lifecycle for the ElkaTech Launchpad platform

This document specifies the PostgreSQL database design for ElkaTech Launchpad. Unlike a purely inferred design, the table definitions here are taken verbatim from the SQL migration files in the repository; anything beyond the committed DDL (sizing guidance, retention policy, future indexes) is explicitly marked as a recommendation. The database is one PostgreSQL instance divided into four service-owned schemas: auth, catalog, service_desk, and notification.

Document	Database Schema Design Document
Product	ElkaTech Launchpad
Version	1.0
Date	June 10, 2026
Status	Approved baseline — matches migrations 001–005
Owner	ElkaTech Engineering / Database
Audience	Backend engineers, DBAs, reviewers
Classification	Internal / Confidential
Source of truth	services/*/migrations/*.sql (verbatim DDL)

Contents

1. Database Overview	4
2. Schema Boundaries	4
3. Entity Relationship Overview	5
4. Table Designs — auth schema	6
4.1 auth.users	6
4.2 auth.sessions	7
4.3 auth.tokens	7
4.4 auth.oauth_identities	8
4.5 auth.outbox	8
5. Table Designs — catalog schema	9
5.1 catalog.categories	9
5.2 catalog.products	9
6. Table Designs — service_desk schema	10
6.1 service_desk.requests	10
6.2 service_desk.request_messages	11
6.3 service_desk.request_history	12
6.4 service_desk.outbox	12
7. Table Designs — notification schema	13
7.1 notification.deliveries	13
8. Enumerations and Status Definitions	13
8.1 User roles (auth.users.role)	14
8.2 Approval statuses (auth.users.approval_status)	14
8.3 Account status and origin (auth.users.status / account_origin)	14
8.4 Request statuses (service_desk.requests.status)	14
8.5 Request priorities (service_desk.requests.priority)	15
8.6 Message visibility (request_messages.visibility)	15
8.7 Outbox event types	15
8.8 Delivery status (notification.deliveries.status)	15
9. User Removal: Soft-Delete Design and Current Behavior	15

9.1 Soft-delete columns (migration 005)	15
9.2 Current behavior: hard delete	16
9.3 Cleanup script ordering (FK-safe)	16
10. Request Workflow State Machine	16
11. Migration Notes	17
12. Seed Data Notes	18
13. Data Retention and Cleanup Strategy	18
14. Security Considerations for Sensitive Fields	19
15. Future Schema Improvements	19
16. Assumptions and Open Questions	19

1. Database Overview

One PostgreSQL database serves the whole platform — Dockerized Postgres locally, Neon (serverless Postgres) in production via Vercel's storage integration. Logical separation is achieved with four schemas, each owned and migrated by exactly one service.

Property	Value
Engine	PostgreSQL (local Docker; Neon hosted in production)
Connection	POSTGRES_URL (production, pooled via pgbouncer) with DATABASE_URL fallback (local: postgres://elkatech:elkatech@127.0.0.1:5432/elkatech)
Pooling note	Prepared statements are disabled on the pgbouncer pooled connection to avoid stale-plan errors
Schemas	auth, catalog, service_desk, notification
Migrations	Plain SQL files per service, applied in order by scripts/db-migrate.ts (npm run db:migrate); written idempotently
Identifiers	uuid primary keys generated by the application (no DB-side default); catalog uses human-readable text ids
Timestamps	timestampz with default now() for created_at / updated_at audit columns
JSON	jsonb for snapshots, payloads, metadata, and catalog arrays (images, specs, highlights)

2. Schema Boundaries

Each schema is private to its owning service; other services reach the data only through that service's internal HTTP API. The single deliberate exception is the notification service, which reads the two outbox tables and the platform-level cleanup/removal scripts, which operate cross-schema inside one transaction.

Schema	Owning service	Contents
auth	services/auth	users, sessions, tokens, oauth_identities, outbox
catalog	services/catalog	categories, products (seeded from packages/contracts)
service_desk	services/service-desk	requests, request_messages, request_history, outbox
notification	services/notification	deliveries (email audit log)

- No cross-schema foreign keys exist: service_desk.requests.customer_id refers to auth.users.id by convention, not by constraint, preserving service autonomy. Integrity across schemas is maintained by the services and by transactional scripts.
- Within a schema, foreign keys with on delete cascade keep dependents (sessions, tokens, identities, messages, history) consistent.

3. Entity Relationship Overview

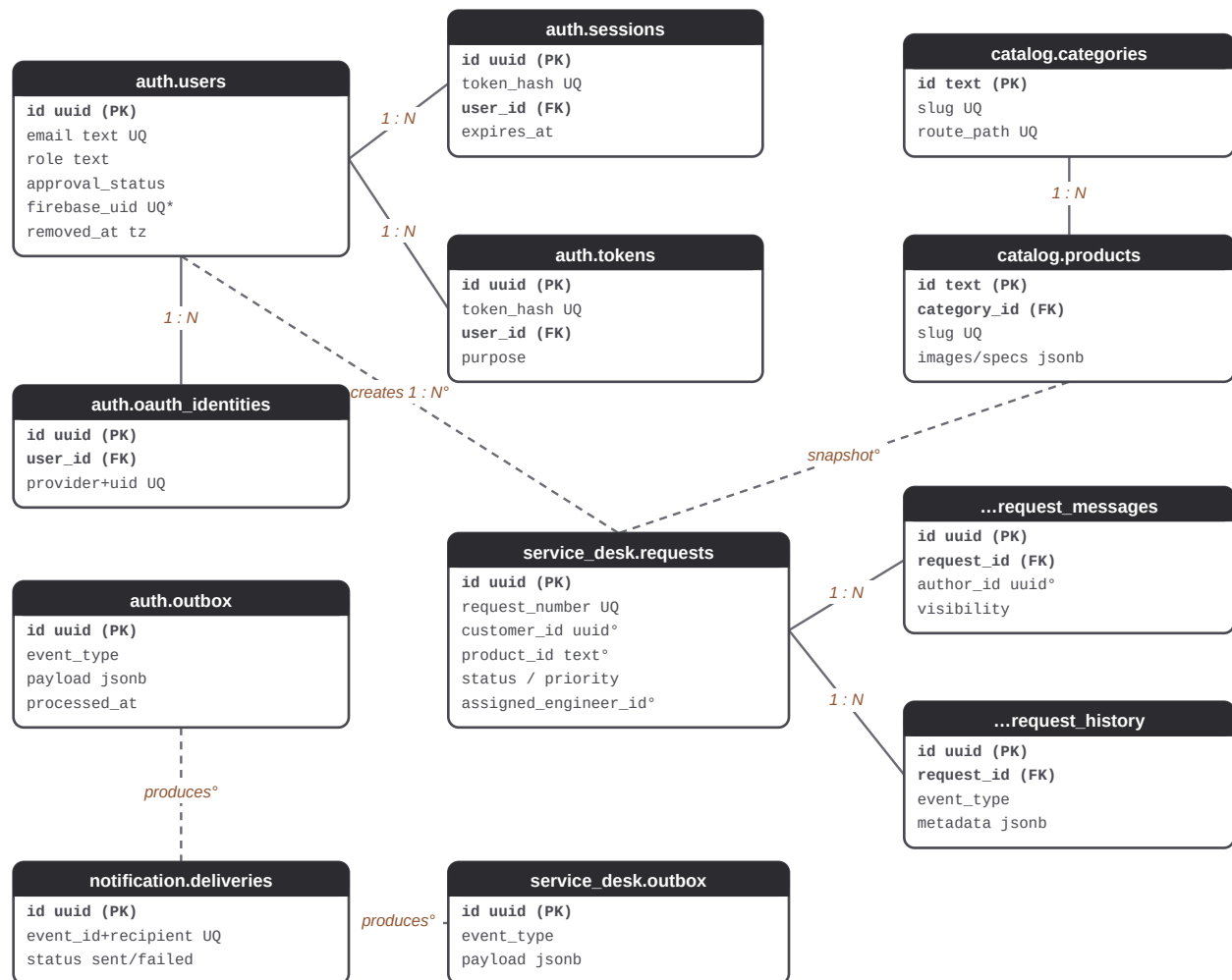


Figure 1 — Entity relationships. Dashed edges marked ° are by-convention references across schema boundaries (no SQL foreign key). UQ* = partial unique index.

Mermaid source for the same model:

```
erDiagram
    AUTH_USERS ||--o{ AUTH_SESSIONS : owns
    AUTH_USERS ||--o{ AUTH_TOKENS : receives
    AUTH_USERS ||--o{ OAUTH_IDENTITY : links
    AUTH_USERS ||--o{ SERVICE_REQUEST : creates
    CATALOG_CATEGORIES ||--o{ CATALOG_PRODUCTS : contains
    SERVICE_REQUESTS ||--o{ REQUEST_MESSAGES : has
    SERVICE_REQUESTS ||--o{ REQUEST_HISTORY : records
    AUTH_OUTBOX ||--o{ NOTIFICATION_DELIVERIES : produces
    SERVICE_DESK_OUTBOX ||--o{ NOTIFICATION_DELIVERIES : produces
```

Listing 1 — ER overview, Mermaid source.

4. Table Designs — auth schema

4.1 auth.users

Purpose. The platform account record: identity, role, approval workflow state, account origin, and removal markers. Created by legacy signup, Firebase session bridge, invite acceptance, or the bootstrap-admin script.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key (app-generated)
email	text	No	unique
display_name	text	No	—
role	text	No	check in ('customer','engineer','admin')
password_hash	text	Yes	Null for Firebase/OAuth-only accounts
email_verified	boolean	No	default false
status	text	No	default 'active'; check in ('active','invited')
firebase_uid	text	Yes	Partial unique index where not null (003)
approval_status	text	No	default 'approved'; check in ('pending_approval','approved','rejected','suspended') (003)
approved_at / approved_by	timestampz / uuid	Yes	Set on approval (003)
rejected_at / rejected_by	timestampz / uuid	Yes	Set on rejection (003)
suspended_at / suspended_by	timestampz / uuid	Yes	Set on suspension (003)
account_origin	text	No	default 'self_signup'; check in ('self_signup','admin_invite','firebase_google','legacy') (004)
removed_at / removed_by	timestampz / uuid	Yes	Soft-delete markers (005); partial index where removed_at is null
created_at / updated_at	timestampz	No	default now()

■ **Indexes:** PK btree on id; unique on email; auth_users_firebase_uid_idx unique partial on firebase_uid where not null; auth_users_removed_at_idx partial on removed_at where removed_at is null.

■ **Note on defaults:** approval_status defaults to 'approved' at the column level (so the 003 backfill kept pre-existing users usable); the application explicitly sets 'pending_approval' for new public signups.

```

id          7c9e6679-7425-40de-944b-e07fc1f90ae7
email       rajan@printworks.example
display_name Rajan Mehta
role        customer
password_hash NULL (Firebase-managed credential)
email_verified true
status      active
firebase_uid fb_uid_8sK2...
approval_status approved
approved_at 2026-05-12 09:14:03+00 approved_by 11d3...(admin id)
account_origin self_signup
removed_at  NULL removed_by NULL
created_at  2026-05-11 16:02:51+00

```

Example row — `auth.users`.

4.2 auth.sessions

Purpose. Server-side cookie sessions. The browser holds an opaque token; only its hash is stored. Each session carries its own CSRF token.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key
token_hash	text	No	unique — hash of the cookie value
csrf_token	text	No	Pairs with the CSRF cookie
user_id	uuid	No	FK → <code>auth.users(id)</code> on delete cascade
user_agent	text	Yes	—
ip_address	text	Yes	—
expires_at	timestamptz	No	<code>now + SESSION_TTL_HOURS</code>
created_at / last_seen_at	timestamptz	No	default <code>now()</code>

■ **Indexes:** PK on `id`; unique on `token_hash` (the lookup path).

■ **Lifecycle:** deleted on logout; all of a user's sessions are deleted on suspension and on account removal; cascade removes sessions if the user row is deleted.

```

id          3f2b... token_hash sha256:9af1... csrf_token c1a7...
user_id      7c9e6679-7425-40de-944b-e07fc1f90ae7
user_agent   Mozilla/5.0 (Macintosh...) ip_address 203.0.113.7
expires_at   2026-06-17 09:14:03+00 created_at 2026-06-10 09:14:03+00

```

Example row — `auth.sessions`.

4.3 auth.tokens

Purpose. Single-use, expiring tokens for the legacy flows: email verification, password reset, and staff invitations. Invite tokens may exist before the user does, so `user_id` is nullable and the target email/role ride on the token.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key

Column	Type	Null	Default / constraint
token_hash	text	No	unique
user_id	uuid	Yes	FK → auth.users(id) on delete cascade; null for not-yet-accepted invites
email	text	No	Target address
role	text	Yes	check in ('customer','engineer','admin'); used by invite tokens
purpose	text	No	check in ('verify_email','reset_password','invite_signup')
expires_at	timestamptz	No	—
consumed_at	timestamptz	Yes	Set when used (single-use)
created_at	timestamptz	No	default now()

```
id      a91c...  token_hash sha256:77b3...  purpose invite_signup
user_id NULL    email dev.engineer@elkatech.example  role engineer
expires_at 2026-06-13 12:00:00+00    consumed_at NULL
```

Example row — auth.tokens (pending engineer invite).

4.4 auth.oauth_identities

Purpose. Links external OAuth identities (the legacy server-side Google flow) to local users; one user may link several providers, and a provider identity maps to exactly one user.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key
user_id	uuid	No	FK → auth.users(id) on delete cascade
provider	text	No	e.g. 'google'
provider_user_id	text	No	Subject from the provider
provider_email	text	No	—
created_at / updated_at	timestamptz	No	default now()

■ **Constraints:** unique (provider, provider_user_id).

```
id 58aa...  user_id 7c9e...  provider google
provider_user_id 103745923847572...  provider_email rajan@gmail.example
```

Example row — auth.oauth_identities.

4.5 auth.outbox

Purpose. Transactional outbox for auth-domain events, written atomically with the state change and polled by the notification service.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key

Column	Type	Null	Default / constraint
aggregate_type	text	No	e.g. 'user'
aggregate_id	text	No	Id of the subject row
event_type	text	No	user.registered, user.email_verified, user.password_reset_requested, user.approval_changed
payload	jsonb	No	Event facts (emails, names, flags)
occurred_at	timestamptz	No	default now()
processed_at	timestamptz	Yes	Null until handled

```
id 0d44... aggregate_type user aggregate_id 7c9e...
event_type user.registered
payload {"email":"dev.engineer@elkatech.example","invitation":true,
        "role":"engineer"}
occurred_at 2026-06-10 12:01:22+00 processed_at 2026-06-10 12:01:25+00
```

Example row — *auth.outbox* (drives the SES account-added template).

5. Table Designs — catalog schema

5.1 catalog.categories

Purpose. Machine families shown in the portal product selector; seeded from packages/contracts. Human-readable text ids keep seed data stable across environments.

Column	Type	Null	Default / constraint
id	text	No	Primary key (seed-stable)
slug	text	No	unique
route_path	text	No	unique — public route, e.g. '/uv-printers'
name	text	No	Display name
intro	text	No	Category intro copy
created_at / updated_at	timestamptz	No	default now()

```
id uv-printers slug uv-printers route_path /uv-printers
name UV Printers intro "High-cure-speed UV printing systems..."
```

Example row — *catalog.categories*.

5.2 catalog.products

Purpose. Sellable machines with display data for the portal selector and request snapshots. Pricing is display-only text (priceDisplay) — there is no transactional commerce.

Column	Type	Null	Default / constraint
id	text	No	Primary key (seed-stable)
category_id	text	No	FK → catalog.categories(id) on delete cascade
category_slug	text	No	Denormalized for filtering
slug	text	No	unique
name	text	No	—
price_display	text	No	Display string, not a numeric price
brochure_url	text	Yes	—
images	jsonb	No	default '[]' — array of image paths
specs	jsonb	No	default '[]' — array of label/value pairs
highlights	jsonb	No	default '[]'
created_at / updated_at	timestampz	No	default now()

```
id uv-printers/falcon-2513  category_id uv-printers
slug falcon-2513  name Falcon 2513 Flatbed UV Printer
price_display "Contact for pricing"  brochure_url /brochures/falcon.pdf
images ["/images/falcon-1.jpg", "/images/falcon-2.jpg"]
specs [{"label": "Print area", "value": "2.5 × 1.3 m"}, ...]
```

Example row — catalog.products (illustrative values).

6. Table Designs — service_desk schema

6.1 service_desk.requests

Purpose. The service request aggregate root. Carries a frozen jsonb product snapshot so later catalog edits never change what the customer reported against. customer_id and assigned_engineer_id reference auth.users by convention (no cross-schema FK).

Column	Type	Null	Default / constraint
id	uuid	No	Primary key
request_number	text	No	unique — human-facing number
customer_id	uuid	No	auth.users id (by convention)
customer_email	text	No	Denormalized for notifications
product_id	text	No	catalog.products id (by convention)
product_snapshot	jsonb	No	Frozen product at creation
subject	text	No	—
description	text	No	—
contact_phone	text	No	—

Column	Type	Null	Default / constraint
site_location	text	No	—
serial_number	text	Yes	Machine serial (optional)
priority	text	No	check in ('low','normal','high','urgent')
status	text	No	check in ('new','triaged','assigned','in_progress','waiting_for_customer','resolved','closed')
assigned_engineer_id	uuid	Yes	Set by claim/assign only
created_at / updated_at	timestampz	No	default now()

Recommended inferred design. The committed DDL defines only the PK and the unique request_number index. Recommended additional indexes based on the read paths: (customer_id, created_at desc) for customer lists, (status) or (status, assigned_engineer_id) for the queue, and (assigned_engineer_id) for engineer scopes — add once row counts make sequential scans noticeable.

```
id 9be0... request_number SR-2026-00042
customer_id 7c9e... customer_email rajan@printworks.example
product_id uv-printers/falcon-2513
product_snapshot {"name":"Falcon 2513...","categorySlug":"uv-printers",...}
subject UV lamp not igniting after warm-up
description Machine powers on but the UV lamp fails to ignite...
contact_phone +91 98765 43210 site_location Pune plant, Bay 3
serial_number F25-13-0117 priority high status in_progress
assigned_engineer_id 4f1d... created_at 2026-06-08 10:22:41+00
```

Example row — service_desk.requests.

6.2 service_desk.request_messages

Purpose. The conversation thread on a request, including internal staff notes. Visibility controls customer exposure; author_role is denormalized so rendering never needs an auth lookup.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key
request_id	uuid	No	FK → service_desk.requests(id) on delete cascade
author_id	uuid	No	auth.users id (by convention)
author_role	text	No	check in ('customer','engineer','admin')
visibility	text	No	check in ('customer_visible','internal_note')
body	text	No	—
created_at	timestampz	No	default now()

```
id 77f2... request_id 9be0... author_id 4f1d... author_role engineer
visibility internal_note
body Lamp driver board shows error E7 — bringing a spare on Friday.
created_at 2026-06-09 15:40:12+00
```

Example row — `service_desk.request_messages` (hidden from the customer).

6.3 `service_desk.request_history`

Purpose. Append-only audit trail per request: who did what, when, with structured metadata. Backs the activity timeline in the request detail UI (newest first, capped at 5 with show-all).

Column	Type	Null	Default / constraint
<code>id</code>	<code>uuid</code>	No	Primary key
<code>request_id</code>	<code>uuid</code>	No	FK → <code>service_desk.requests(id)</code> on delete cascade
<code>actor_id</code>	<code>uuid</code>	No	<code>auth.users id</code> (by convention)
<code>actor_role</code>	<code>text</code>	No	check in ('customer','engineer','admin')
<code>event_type</code>	<code>text</code>	No	<code>request_created</code> , <code>request_claimed</code> , <code>request_assigned</code> , <code>request_reassigned</code> , <code>status_changed</code> , <code>request_updated</code> , <code>request_archived</code> , <code>request_cancelled</code>
<code>metadata</code>	<code>jsonb</code>	No	default '{}' — e.g. {from,to} for <code>status_changed</code> ; {previousEngineerId, engineerId, engineerEmail} for <code>request_reassigned</code>
<code>created_at</code>	<code>timestampz</code>	No	default now()

```
id c3aa... request_id 9be0... actor_id 11d3... actor_role admin
event_type request_reassigned
metadata {"previousEngineerId":"4f1d...", "engineerId":"82be...",
         "engineerEmail":"dev.engineer@elkatech.example"}
created_at 2026-06-09 17:05:33+00
```

Example row — `service_desk.request_history`.

6.4 `service_desk.outbox`

Purpose. Transactional outbox for request-domain events; identical shape to `auth.outbox`.

Column	Type	Null	Default / constraint
<code>id</code>	<code>uuid</code>	No	Primary key
<code>aggregate_type</code>	<code>text</code>	No	e.g. 'request'
<code>aggregate_id</code>	<code>text</code>	No	Request id
<code>event_type</code>	<code>text</code>	No	<code>request.created</code> , <code>request.assigned</code> , <code>request.status_changed</code> , <code>request.customer_message_posted</code> , <code>request.staff_reply_posted</code>
<code>payload</code>	<code>jsonb</code>	No	Facts for email building (request number, product, emails, names, URLs)
<code>occurred_at</code>	<code>timestampz</code>	No	default now()
<code>processed_at</code>	<code>timestampz</code>	Yes	Null until handled

```
id e51b... aggregate_type request aggregate_id 9be0...
event_type request.assigned
payload {"requestId":"9be0...", "requestNumber":"SR-2026-00042",
        "customerEmail":"rajan@printworks.example",
        "engineerEmail":"dev.engineer@elkatech.example",
        "product":"Falcon 2513...", "location":"Pune plant, Bay 3", ...}
processed_at NULL
```

Example row — *service_desk.outbox* (feeds the SES request-claimed template for both recipients).

7. Table Designs — notification schema

7.1 notification.deliveries

Purpose. Per-recipient email delivery log and idempotency guard. Migration 002 replaced the original `unique(event_id)` with a `unique(event_id, recipient_email)` index so multi-recipient events (e.g. request claimed → customer + engineer) record independently while retries stay idempotent.

Column	Type	Null	Default / constraint
id	uuid	No	Primary key
event_id	uuid	No	Outbox event id (001 had unique; relaxed by 002)
event_type	text	No	Copied from the outbox row
recipient_email	text	No	—
subject	text	No	Rendered subject (or log subject for SES templated sends)
status	text	No	default 'sent'; check in ('sent','failed')
attempts	integer	No	default 1
last_error	text	Yes	Failure detail (SES/SMTP error)
sent_at	timestampz	Yes	Set on success
created_at	timestampz	No	default now()

- **Indexes:** PK on id; `deliveries_event_recipient_uidx` unique on (event_id, recipient_email).
- **No FK** to the outbox tables — deliveries survive outbox cleanup and user removal as an audit log (intentional).

```
id 6a02... event_id e51b... event_type request.assigned
recipient_email rajan@printworks.example
subject "Service request picked up: SR-2026-00042"
status sent attempts 1 last_error NULL
sent_at 2026-06-09 17:05:41+00
```

Example row — *notification.deliveries*.

8. Enumerations and Status Definitions

All enumerations are implemented as text columns with CHECK constraints (not Postgres enum types), which keeps additive migrations simple.

8.1 User roles (auth.users.role)

Value	Meaning
customer	Self-signed-up machine owner/operator; can create and follow own requests once approved
engineer	Invited staff; works the queue, claims requests, posts replies and internal notes, drives status
admin	Invited/bootstrapped staff; everything engineers can do plus approvals, invitations, assignment, reopen, user removal, dashboards

8.2 Approval statuses (auth.users.approval_status)

Value	Set when	Effect
pending_approval	New public customer signup	Can sign in and browse; cannot create requests (USER_PENDING_APPROVAL)
approved	Admin approves; staff invite; legacy backfill	Full access for the role
rejected	Admin rejects	Cannot create requests (USER_REJECTED)
suspended	Admin suspends	Cannot create requests (USER_SUSPENDED); sessions dropped

8.3 Account status and origin (auth.users.status / account_origin)

Field	Values
status	active · invited
account_origin	self_signup (locked to customer role) · admin_invite · firebase_google · legacy

8.4 Request statuses (service_desk.requests.status)

Value	Meaning
new	Submitted, not yet triaged
triaged	Reviewed and categorized by staff
assigned	Owned by an engineer — entered only via claim/assign, never via the status endpoint
in_progress	Actively being worked
waiting_for_customer	Blocked on customer input
resolved	Fix delivered, pending closure
closed	Terminal (admin may reopen to new)

8.5 Request priorities (service_desk.requests.priority)

Value	Intent
low	Cosmetic or non-blocking issues
normal	Default — standard service visit
high	Production impaired
urgent	Production down

Scope. Priorities are informational; no SLA timers are attached to them in the current product.

8.6 Message visibility (request_messages.visibility)

Value	Meaning
customer_visible	Part of the customer-facing conversation
internal_note	Staff-only; never returned to customers; badge-labelled in staff UI

8.7 Outbox event types

Outbox	event_type values
auth.outbox	user.registered · user.email_verified · user.password_reset_requested · user.approval_changed
service_desk.outbox	request.created · request.assigned · request.status_changed · request.customer_message_posted · request.staff_reply_posted

8.8 Delivery status (notification.deliveries.status)

Value	Meaning
sent	Provider accepted the message
failed	Send failed; last_error holds the provider error; attempts counts tries

9. User Removal: Soft-Delete Design and Current Behavior

9.1 Soft-delete columns (migration 005)

- **removed_at timestampz null** — when the account was removed.
- **removed_by uuid null** — the admin who removed it.
- Deliberately distinct from approval_status so a suspended-but-restorable user is never confused with a removed user that should disappear from the accounts list.

- **Filtering:** user listing and summary queries filter *where removed_at is null*; the partial index `auth_users_removed_at_idx` (on rows where `removed_at` is null) keeps that filter cheap.
- **Session cleanup:** removal deletes all of the user's sessions so a removed user is signed out everywhere immediately.
- **Schema tolerance:** the auth service probes `information_schema.columns` once (`hasRemovedAtColumn`) and degrades gracefully if the migration has not been applied, avoiding transaction-aborting 42703 errors.

9.2 Current behavior: hard delete

Current behavior. The soft-delete flow proved insufficient in practice: soft-removed users resurfaced as `SUSPENDED` and their request data was never cleaned. The admin Remove User action now performs a permanent hard delete in a single cross-schema transaction — `service_desk.outbox/messages/history/requests` owned by the user, then `auth.sessions/tokens/oauth_identities/user-scoped outbox events`, then the `auth.users` row — rolling back fully on any failure. All admin accounts (self, system/bootstrap, any role=admin) are protected from removal. Firebase Auth deletion runs best-effort after commit. The `removed_at/removed_by` columns remain in the schema; whether to keep them as a safety net or retire them is an open decision.

9.3 Cleanup script ordering (FK-safe)

```
service_desk.outbox      -- user-scoped events
service_desk.request_messages
service_desk.request_history
service_desk.requests
auth.sessions
auth.tokens
auth.oauth_identities    -- when present
auth.outbox              -- aggregate_type='user' AND aggregate_id IN (...)
auth.users               -- last; admins always kept
```

Listing 2 — Deletion order used by user removal and `scripts/cleanup-test-data.ts` (single transaction). `notification.deliveries` is intentionally retained as an audit log; Firebase cleanup is a documented manual follow-up for the bulk script.

10. Request Workflow State Machine

The status column is governed by a strict transition map enforced in `services/service-desk/src/workflow.ts` and mirrored in the frontend. 'assigned' is set exclusively by claim/assign; same-status updates are rejected; reopening from resolved/closed to new is admin-only.

From \ To	new	triaged	assigned	in_progress	waiting_for_customer	resolved	closed
new	—	Yes	claim/assign	.	.	.	Yes
triaged	.	—	claim/assign	Yes	Yes	Yes	Yes
assigned	.	Yes	—	Yes	Yes	Yes	Yes
in_progress	.	.	.	—	Yes	Yes	Yes
waiting_for_customer	.	.	.	Yes	—	Yes	Yes

From \ To	new	triaged	assigned	in_progress	waiting_for_customer	resolved	closed
resolved	admin	.	.	.	.	—	Yes
closed	admin	.	.	.	.	.	—

“claim/assign” marks entry into assigned via the dedicated endpoints rather than the /status route; “admin” marks the admin-only reopen; “.” is disallowed.

```
stateDiagram-v2
    [*] --> new
    new --> triaged
    new --> closed
    new --> assigned : claim / assign
    triaged --> assigned : claim / assign
    triaged --> in_progress
    triaged --> waiting_for_customer
    triaged --> resolved
    triaged --> closed
    assigned --> triaged
    assigned --> in_progress
    assigned --> waiting_for_customer
    assigned --> resolved
    assigned --> closed
    in_progress --> waiting_for_customer
    in_progress --> resolved
    in_progress --> closed
    waiting_for_customer --> in_progress
    waiting_for_customer --> resolved
    waiting_for_customer --> closed
    resolved --> new : admin reopen
    resolved --> closed
    closed --> new : admin reopen
```

Listing 3 — Workflow state machine, Mermaid source.

11. Migration Notes

Migration	Purpose
auth/001_init	auth schema; users, sessions, tokens, outbox with role/status/purpose CHECKs
auth/002_oauth_identities	OAuth identity linking with unique (provider, provider_user_id)
auth/003_firebase_and_approval	firebase_uid (partial unique), approval_status + actor/timestamp columns, guarded CHECK; backfills existing users to approved
auth/004_account_origin	account_origin with CHECK; backfills staff → admin_invite, old customers → legacy
auth/005_user_removed_at	removed_at / removed_by plus partial index on live rows
catalog/001_init	catalog schema; categories and products with jsonb arrays
service-desk/001_init	service_desk schema; requests, messages, history, outbox with status/priority/visibility CHECKs
notification/001_init	notification schema; deliveries with unique event_id

Migration	Purpose
notification/002_deliveries_recipient_unique	Drops unique(event_id); adds unique (event_id, recipient_email) for multi-recipient events

- Migrations are applied in filename order per service by scripts/db-migrate.ts; all are idempotent (if-not-exists guards, conditional constraint creation) so re-running is safe.
- Production: run once from an operator machine — `POSTGRES_URL='<neon-pooled-url>' npm run db:migrate` (then `db:seed`, `db:bootstrap-admin`). Single-quote the URL; it contains ampersands.
- Preview deployments share the production database (Neon branch-per-deployment disabled) — never test destructive migrations via previews.

12. Seed Data Notes

- `npm run db:seed` loads `catalog.categories` and `catalog.products` from `catalogSeedData` in `packages/contracts/src/index.ts` — the portal catalog's source of truth.
- Public category pages render from separate local component arrays; the two datasets are parallel and must be updated together when products change (documented constraint).
- `npm run db:bootstrap-admin` creates or updates the initial admin from `BOOTSTRAP_ADMIN_EMAIL` / `BOOTSTRAP_ADMIN_PASSWORD`, and also provisions the admin in Firebase Auth when Admin SDK env vars are set.
- Text PKs in catalog keep seeds re-runnable: re-seeding upserts stable ids rather than generating duplicates.
- `scripts/cleanup-test-data.ts` (`npm run db:cleanup-test-data`) removes non-admin users and their data; dry-run by default, destructive only with `-- --confirm`, and aborts if zero admins would remain.

13. Data Retention and Cleanup Strategy

Data	Retention behavior
Sessions	Bounded by <code>expires_at</code> (<code>SESSION_TTL_HOURS</code>); deleted on logout/suspension/removal; cascade on user delete
Auth tokens	Single-use (<code>consumed_at</code>) with <code>expires_at</code> ; cascade on user delete
Outbox rows	Marked <code>processed_at</code> after handling; retained thereafter (no automatic purge defined)
Deliveries	Retained indefinitely as the email audit log; intentionally no FK so they survive cleanup
Requests / messages / history	Retained for the life of the customer account; permanently deleted with the owning user on removal
Removed users	Hard-deleted (current behavior); admin accounts protected

Recommended inferred design. No scheduled purge jobs exist yet. Recommended policy to adopt: delete expired sessions and consumed/expired tokens on a daily schedule; archive or purge processed outbox rows after 90 days; review delivery-log retention against privacy obligations (recipient emails are personal data).

14. Security Considerations for Sensitive Fields

Field(s)	Consideration
auth.users.password_hash	Only a hash is stored (legacy accounts); null for Firebase/OAuth accounts — no plaintext credentials anywhere
auth.sessions.token_hash / auth.tokens.token_hash	Raw tokens never stored; lookups by hash; cookies and tokens must never be logged
auth.sessions.csrf_token	Server-side anchor for the double-submit CSRF check
auth.sessions.ip_address / user_agent	Personal data — expose only to the session owner and admins; include in deletion scope
auth.users.firebase_uid	External identity key; partial unique index; treat as opaque
requests.contact_phone / site_location / customer_email	Customer PII — visibility limited to the owning customer and staff; deleted with the user
request_messages (internal_note)	Must never cross the visibility boundary; enforced in service queries, not just UI
notification.deliveries.recipient_email	PII retained as audit log — document retention period and lawful basis
outbox payloads	Carry emails/names; treat as PII when designing retention/purges

15. Future Schema Improvements

- Add the recommended read-path indexes on service_desk.requests (customer, status/queue, assignee) once volume warrants.
- Decide the fate of removed_at/removed_by now that removal is a hard delete: keep as safety net (and re-introduce a restorable archive flow) or drop in a future migration.
- Consider a request_number sequence/generator table if the current generation scheme ever needs gap-free or per-year numbering guarantees.
- Add updated_at triggers (or application discipline) so updated_at is always touched on mutation.
- Introduce scheduled cleanup (sessions, tokens, processed outbox) as a maintenance script or cron.
- If attachments are added to requests/messages, store object-store references (not blobs) in a new table with per-request FKs.
- Single-source the catalog so catalog.* is generated from the same data as the public pages.

16. Assumptions and Open Questions

- **Verbatim DDL.** Sections 4–7 reproduce the committed migrations exactly; example rows are illustrative (fabricated values, realistic shapes).
- **Request numbering.** request_number is unique text; the exact generation format (e.g. SR-YYYY-NNNNN) is application-defined and the example is illustrative.
- **History event payloads.** Metadata shapes are confirmed for reassignment and status changes; other event metadata follows the service implementation.
- **Retention policy.** All purge schedules in section 13 are recommendations pending business/privacy sign-off.
- **Outbox growth.** No partitioning is needed at current volumes; revisit if event volume grows by orders of magnitude.